API Gateway Security

Explore how to secure AWS API Gateway endpoints through layered access controls including IAM roles, resource policies, token-based authorizers, and network protections. Understand securing REST, HTTP, and WebSocket APIs, managing TLS certificates, and configuring CORS for browser access. This lesson helps you apply robust security best practices to protect APIs in serverless computing.

Once an API is deployed and observable, securing it becomes critical. API Gateway offers multiple mechanisms to protect endpoints against unauthorized or unintended access. These mechanisms can be layered together to form a robust access control strategy depending on the API type (HTTP, REST, and WebSocket) and its intended consumers.

We'll now explore each mechanism individually, organized by how they integrate with IAM, token-based identity, WebSocket connection control, and network-level protection.

IAM-based authorization

For internal applications or backend services hosted in AWS, IAM provides native access control to API Gateway. IAM policies can be used to define fine-grained access to specific resources or HTTP methods.

Method-level authorization

When securing our API Gateway, we're not limited to global access contro

Ask

Method-level authorization lets us apply different permissions to individual resources or HTTP methods (like, `GET /items`, `POST /items`). This works in

conjunction with IAM policies and resource policies to offer fine-grained access control.

For example, we can attach an IAM policy that allows a specific role to invoke only the `GET` method on a specific resource path:

main.ml C++

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": "execute-api:Invoke",
      "Resource": "arn:aws:execute-api:us-east-1:123456789012:abc123/
    }
  ]
}
```

Sample IAM policy

In cross-account scenarios, the IAM role assumed by a different AWS account must also have permissions to invoke the method, and the API's resource policy must allow the caller's AWS account.

Similarly, when API Gateway integrates with a Lambda function, it invokes the function, but the Lambda executes under its own IAM role. This role governs what the function can access, such as reading from DynamoDB or writing to S3.

> **Exam tip:** Understanding this boundary helps reinforce the principle of privilege and is often tested in scenario-based questions.

Ask

Resource policies

Resource policies act as access control lists for API Gateway resources. We can restrict access based on source IP, AWS account, or VPC endpoint. These policies are necessary for enabling cross-account access or limiting exposure to the public internet.

Let's consider a scenario where we want to allow a partner account to call our API, but only from a specific IP range. We can use a resource policy like the one below:

index.yaml YAML

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowFromSpecificAccountAndIP",
      "Effect": "Allow",
      "Principal": "*",
      "Action": "execute-api:Invoke",
      "Resource": "arn:aws:execute-api:us-east-1:123456789012:abc123/
      "Condition": {
        "IpAddress": {
          "aws:SourceIp": "203.0.113.0/24"
        }
```

Resource policy document

This policy ensures that only principals from AWS account `111122223333` who are calling from the IP range `203.0.113.0/24` can invoke our API. It's a common pattern for securing partner-facing APIs.

To apply this policy to a REST API using the AWS CLI, we'll first save it to file and the use the `update-rest-api` command to apply it:

Ask

main.sh Shell

```
# Save the policy
cat > policy.json <<EOF
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowFromSpecificAccountAndIP",
      "Effect": "Allow",
      "Principal": "*",
      "Action": "execute-api:Invoke",
      "Resource": "arn:aws:execute-api:us-east-1:123456789012:abc123/
      "Condition": {
```

Apply resource-based policy

Once applied, this resource policy becomes an additional layer of enforcement. Even if a user has valid credentials and passes IAM-based checks, they'll be denied access if they don't meet the policy's network or account conditions.

Token-based authorizers

APIs accessed by frontend clients, mobile apps, or third-party integrations often use bearer tokens to authenticate users. This approach allows us to control access based on the identity of the requester, without relying on IAM roles or resource policies.

Lambda authorizers

Lambda authorizers offer customizable access control. These are Lambda functions that evaluate incoming requests and return IAM policy documents. We can implement token-based authorizers that handle OAuth or JWT tokens, or request-based authorizers that evaluate headers and query strings.

Ask

We can create the authorizer by linking it to a Lambda function:

main.sh C++

```
aws apigateway create-authorizer \
  --rest-api-id $REST_API_ID \
  --name CustomAuth \
  --type TOKEN \
  --authorizer-uri arn:aws:apigateway:us-west-2:lambda:path/2015-03-3
```

Create a Lambda authorizer

This is useful for implementing custom authentication logic or integrating with third-party identity providers. Lambda authorizers return an IAM policy that defines what the user is allowed (or denied) to invoke. Here's a simplified structure of what our Lambda authorizer must return:

main.sh Shell

```
{
  "principalId": "user|a1b2c3d4",
  "policyDocument": {
    "Version": "2012-10-17",
    "Statement": [
      {
        "Action": "execute-api:Invoke",
        "Effect": "Allow",
        "Resource": "arn:aws:execute-api:us-east-1:123456789012:xyz78
      }
    ]
  },
  "context": {
```

Ask

IAM policy returned by the authorizer

The above response grants the caller permission to invoke a specific API method. The `Effect` is set to `Allow`, which explicitly permits the action. The `Action`, `execute-api:Invoke`, authorizes the caller to invoke API Gateway methods. The `Resource` ARN scopes this permission to the `GET` method on the `/items` resource of the API with ID `xyz789` in the `prod` stage. This fine-grained permission ensures the caller can only access this specific method and resource, preventing access to other API endpoints.

The `context` object is optional but allows custom key-value pairs to be passed to the backend Lambda integration.

Amazon Cognito authorizers

Cognito authorizers validate tokens issued by Amazon Cognito User Pools. This is ideal for user-facing applications with built-in registration, sign-in, and multi-factor authentication. To set up cognito authorizers, we first create the authorizer:

main.sh C++

```
aws apigateway create-authorizer \
  --rest-api-id $REST_API_ID \
  --name CognitoAuth \
  --type COGNITO_USER_POOLS \
  --provider-arns arn:aws:cognito-idp:us-west-2:123456789012:userpool/
```

Ask

Create Cognito authorizer

Then, we attach it to a method:

main.sh C++

```
aws apigateway update-method \
  --rest-api-id $REST_API_ID \
  --resource-id $RESOURCE_ID \
  --http-method GET \
  --authorization-type COGNITO_USER_POOLS \
  --authorizer-id $AUTHORIZER_ID
```

Attach authorizer to a method

Now when a user signs in, Cognito issues an ID token, which is passed in the `Authorization` header. API Gateway validates the token and grants access if the token is valid and not expired.

Authenticating WebSocket APIs

In WebSocket APIs, authentication is typically handled at the `$connect` route using custom logic within the integration Lambda function. This function inspects the request headers or query parameters for credentials such as JWT tokens or API keys and performs validation before allowing the connection to persist.

Here's a typical flow:

1. The client initiates a WebSocket connection and includes an authenti token in the query string or headers.
2. API Gateway invokes the `$connect` integration, usually a Lambda function.

Ask

3. The function parses and validates the token.
4. If the token is invalid, the function returns an error (e.g., 403), causing the connection to be rejected.

Example of checking a query string token in the `$connect` Lambda function:

main.py Python 3.10.4

```
def lambda_handler(event, context):
    params = event.get('queryStringParameters', {}) or {}
    token = params.get('authToken', '')

    if not validate_token(token):
        return {
            'statusCode': 403,
            'body': 'Unauthorized'
        }

    return {
        'statusCode': 200,
        'body': 'Connection accepted'
```

Lambda function code with embedded authentication

For network-level access control, we can still apply IAM policies and resource policies to the WebSocket API, such as restricting access by source IP or AWS account. However, all fine-grained user validation must be performed manually in our `$connect` logic.

TLS enforcement and ACM integration

All API Gateway endpoints use HTTPS by default. When using a custom domain, we must associate a TLS certificate via AWS Certificate Manager (ACM). ACM automates certificate renewals and supports wildcard domains. We bind the domain to our API through base path mappings, allowing us to route requests to different stages under the same domain.

Ask

Note that TLS enforcement and custom domain setup differ slightly between REST and HTTP APIs. For REST APIs, we can configure base path mappings and domain names directly in API Gateway. For HTTP APIs, the process is more streamlined, but some features, like mutual TLS, may require additional setup. In both cases, certificates are provisioned through ACM and bound to the domain name to ensure secure communication.

Configuring CORS for REST and HTTP APIs

When building web applications that interact with API Gateway, Cross-Origin Resource Sharing (CORS) often becomes a hurdle. Browsers enforce CORS policies that restrict requests from different origins unless explicitly allowed by the server. Without proper configuration, even valid API calls can fail with errors like `403 Forbidden` or `CORS policy: No 'Access-Control-Allow-Origin' header`.

In REST APIs, enabling CORS requires manually configuring each method to return appropriate headers. We typically add an `OPTIONS` method to handle preflight requests and set response headers like `Access-Control-Allow-Origin`, `Access-Control-Allow-Methods`, and `Access-Control-Allow-Headers`. This can be done through the console or CLI using method response and integration response mappings.

main.sh Shell

```shell
aws apigateway put-method-response \
  --rest-api-id <rest-api-id> \
  --resource-id <resource-id> \
  --http-method OPTIONS \
  --status-code 200 \
  --response-parameters "{\"method.response.header.Access-Control

aws apigateway put-integration-response \
  --rest-api-id <rest-api-id> \
  --resource-id <resource-id> \
```

Ask

```
  --http-method OPTIONS \
  --status-code 200 \
```

Handle preflight requests and set response headers

In HTTP APIs, the process is simpler. CORS is handled as a first-class configuration at the route level. We can specify allowed origins, headers, and methods during API creation or later via `update-api`. Here's an example using the AWS CLI:

main.sh Shell

```
aws apigatewayv2 update-api \
  --api-id <http-api-id> \
  --cors-configuration AllowOrigins='["*"]',AllowMethods='["GET","POS
```

Set allowed origins and methods

This declarative model reduces the chance of misconfiguring headers and is recommended when using HTTP APIs for frontend apps.

Regardless of the API type, remember that CORS is a browser-enforced security mechanism, not an AWS authentication control. API Gateway simply respo with the right headers to let the browser allow or block the request.

Ask

> **Note:** WebSocket APIs do not support CORS. Because WebSockets aren't subject to the same origin policies, CORS is unnecessary and unsupported.

Conclusion

AWS API Gateway provides a layered set of access control mechanisms, from IAM roles and resource policies to Lambda and Cognito authorizers, each suited to different use cases. Whether we're building internal APIs for trusted AWS clients or public-facing endpoints for web and mobile apps, these tools allow us to enforce the right level of access with precision. Understanding how and when to use each mechanism ensures our API remains both accessible and secure, no matter the scale or audience.

Ask